

CS 3333 Final Exam (Review)

1. (Multiple choices) [10 points] Consider any given undirected unweighted graph $G(V, E)$. The diameter of G is defined as the maximum shortest distance between any pair of vertices u and v . Which of the following will find the diameter of G ?
 - 1) Call Dijkstra's algorithm starting with any given vertex u to find a shortest path to every other vertex. The longest path from u to some vertex is the diameter of G .
 - 2) Call Prim's algorithm starting with any given vertex u to find a minimum spanning tree. The longest path from u to some vertex is the diameter of G .
 - 3) Call Kruskal's algorithm to find a minimum spanning tree. Pick a root u and the longest path from u to some vertex is the diameter of G .
 - 4) None of the above.

2. (Multiple choices) [10 points] Given a set of points (x_i, y_i) , $1 \leq i \leq n$, we need to find two points with the shortest distance. Obviously, there is an $O(n^2)$ time algorithm for solving this problem. Which of the following is true?
 - 1) With divide and conquer, we can solve this problem in $O(n \log n)$ time.
 - 2) With a greedy algorithm, we can solve this problem in $O(n \log n)$ time.
 - 3) With dynamic programming, we can solve this problem in $O(n \log n)$ time.
 - 4) None of the above.

3. (Multiple choices) [10 points] Given a list of (possible negative) integers $a_1, a_2, \dots, a_n$, find the maximum value of $S(i, j) = a_i + a_{i+1} + \dots + a_j$, $1 \leq i \leq j \leq n$. Note: For convenience, the maximum value is 0, if all integers are negative. Which of the following is the best possible for solving this problem?
 - 1) A 3-nested-loop algorithm with $O(n^3)$ time complexity
 - 2) A 2-nested-loop algorithm with $O(n^2)$ time complexity
 - 3) A divide and conquer algorithm with $O(n \log n)$ time complexity
 - 4) A simple linear scan algorithm with $O(n)$ time complexity.

2. (13 points) Basic Knowledge 1

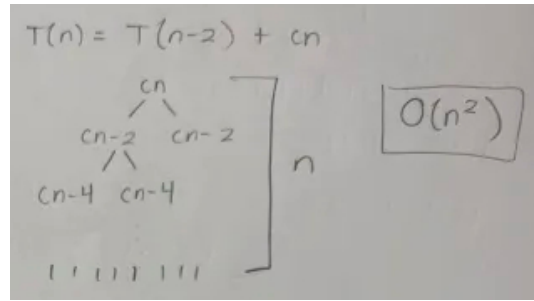
1. What is the big-Oh run time of Merge Sort on n items? $O(n \log n)$
2. What is the big-Oh run time of Heap Sort on n items? $O(n \log n)$
3. What is the big-Omega lower bound for comparison based sorting of n items? $\Omega(n \log n)$
4. What is the worst case big-Oh run time to insert 1 item into an AVL-tree that contains n items? $O(\log n)$
5. What is the worst case big-Oh run time of binary search on an n item sorted list? $O(\log n)$
6. What is the worst case big-Oh run time to insert 1 item into a min-heap? $O(\log n)$
7. What is the worst case big-Oh run time to remove the minimum value from a min-heap? $O(\log n)$
8. What is the run time of breadth-first search (in terms of $|V|$ and $|E|$)? $O(|V| + |E|)$
9. What is the run time of Dijkstra's algorithm (in terms of $|V|$ and $|E|$) assuming a min-heap based implementation? $O((|V| + |E|) \log |V|)$
10. What is the run time of Dijkstra's algorithm (in terms of $|V|$ and $|E|$) assuming a Fibonacci-heap based implementation? $O(|V| \log |V| + |E|)$
11. What is the run time of the Bellman-Ford single-source shortest path algorithm (in terms of $|V|$ and $|E|$)? $O(|V| \cdot |E|)$
12. What is the run time of the Floyd-Warshall "all-pairs" shortest path algorithm (The one that uses dynamic programming)? $O(|V|^3)$
13. What is the run time of the Edmonds-Karp maximum flow algorithm? $O(|V| \cdot |E|^2)$

3. (12 points) Basic Knowledge 2. Circle all of the statements below that are true:

- a. $\log(n) = O(n)$
- b. $n = O(\log n)$
- c. $\log^2(n) = O(n)$
- d. $n = O(\log^2 n)$
- e. $\log(n) = \Omega(n)$
- f. $n = \Omega(\log n)$
- g. $5n^3 + 7n + 13 = O(n^5)$
- h. $5n^3 + 7n + 13 = \Omega(n^5)$
- i. $5n^3 + 7n + 13 = \Omega(n)$
- j. $\log(n!) = \Theta(n \log(n))$
- k. $n^{1/2} = O(\log n)$
- l. $2^n = \Omega(n!)$

4. (20 points) Analyze the big-oh run time for each function with respect to parameter n:

```
void A(int n) O(n2)
{
    if (n < 10)
        time++;
    else
    {
        for (int i = 0; i < n; i++)
            time++;
        A(n - 2);
    }
}
```

$$T(n) = T(n-2) + cn$$


$$O(n^2)$$

```
void B(int n) O(n3)
{
    if (n < 10)
        time++;
    else
    {
        for (int i = 0; i < 8; i++)
            B(n / 2);
        for (int i = 0; i < n*n; i++)
            time++;
    }
}
```

$$T(n) = 8 T\left(\frac{n}{2}\right) + cn^2$$

$$\frac{8}{2^2} > 1 \quad \text{so} \quad O(n^{\log_b a})$$

$$O(n^{\log_8 8^2}) = \boxed{O(n^3)}$$

```
void C(int n) O(n)
{
    if (n < 10)
        time++;
    else
    {
        C(n / 5);
        C(n / 10);
        for (int i = 0; i < n; i++)
            time++;
    }
}
```

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{n}{10}\right) + cn$$

$$1 \cdot \frac{1}{5}^r + 1 \cdot \frac{1}{10}^r = 1 \quad r < 1$$

$$O(n^{\alpha \log_8 8}) = \boxed{O(n)}$$

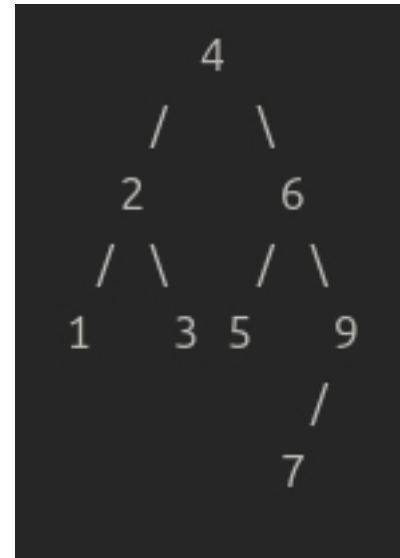
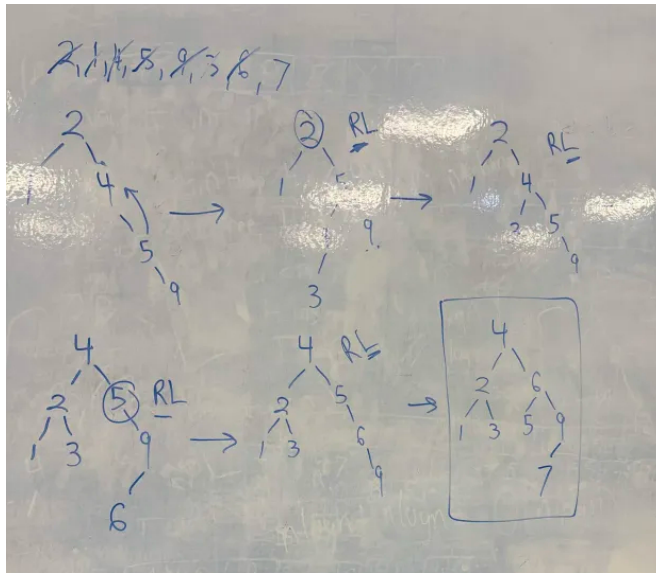
```
void D(int n) O(n)
{
    time = 0;
    for (i = 0; i < n; i++)
        for (j = 0; j < i * i; j++)
            for (k = 0; k < j*j; k++)
                time++;
}
```

$$n * n^2 * n^4 = \boxed{O(n^7)}$$

```
void E(int n) O(n log n)
{
    minHeap H;
    AVLtree T;
    for (int i = 0; i < n; i++)
    {
        H.insert(i);
        T.insert(i);
    }

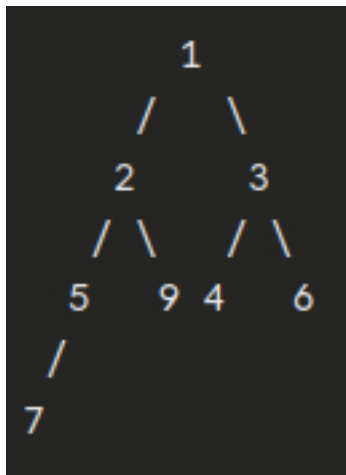
    for (int i = 0; i < n; i++)
        H.extractMin();
}
```

5. (10 points) **AVL Trees:** Show the result of inserting 2,1,4,5,9,3,6,7 into an initially empty AVL tree.

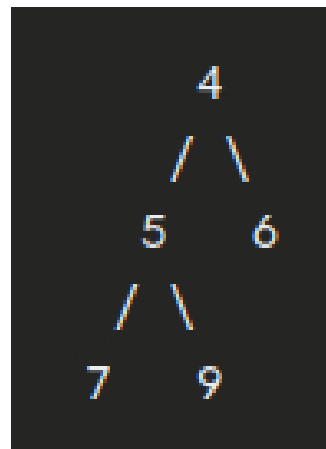


6. (10 points) **Heaps:** Show the result of inserting 2,1,4,5,9,3,6,7 into an initially empty min-heap. Then, perform 3 extract-min operations and show the final resultant heap.

After all insertions



After all extractions



7. (10 points) **Hash Tables:** Show the result of hashing the following values using the hash function $h(x) = x \bmod 10$ and quadratic probing to resolve collisions. The values are: 4371, 1323, 6173, 4199, 4344, 9679, 1989.

Slot	Value	Notes
0	9679	$h=9, +1^2=0$
1	4371	$h=1$
2	—	
3	1323	$h=3$
4	6173	$h=3, +1^2=4$
5	4344	$h=4, +1^2=5$
6	—	
7	—	
8	1989	$h=9, +1^2=0$ taken, $+2^2=3$ taken, $+3^2=8$
9	4199	$h=9$

Coding: Binary Trees

For the following questions, assume binary trees consist of nodes from the following class:

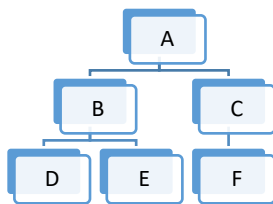
```
class node{
public:
    int data;
    node * left;
    node * right;
};
```

8. (15 points) Write a function such that given a root of tree and an integer sum, return true if there is a path from the root down to a leaf, such that adding up all the values along the path equals the given sum.

```
bool hasPathSum(node* p, int sum) {
    bool hasPathSum(node* p, int sum) {
        if (p == nullptr) return false;
        // Leaf node: check if remaining sum equals this node's value
        if (p->left == nullptr && p->right == nullptr)
            return sum == p->data;
        return hasPathSum(p->left, sum - p->data)
            || hasPathSum(p->right, sum - p->data);
    }
}
```

9. (15 points) Write a method 'levelOrderTraversal(node * r)' which prints the items of a binary tree in a "level order". That is, the first item printed is the root, the next items printed are the children of the root, the next items printed are the grandchildren of the root, etc. You may freely use STL containers such as the vector, stack, or queue data types to solve the problem. Analyze the run time of your solution (more points for a faster solution).

The following tree, for example, would be printed in the following order: A B C D E F



```
void levelOrderTraversal(node* r) {
    if (r == nullptr) return;
    queue<node*> q;
    q.push(r);
    while (!q.empty()) {
        node* curr = q.front(); q.pop();
        cout << curr->data << " ";
        if (curr->left) q.push(curr->left);
        if (curr->right) q.push(curr->right);
    }
}
```

10. (15 points) Coding: Trie Coding

Write a function 'int numEntries(Node * p)' that computes and returns the number of strings stored in the Trie rooted at node p. Use the provided Node class.

```
class Node
{
public:
    //Variable denoting if this node is the end
    //character of a string in the Trie
    bool marked;

    Node * children[256];

    Node()
    {
        marked = false;

        for (int i = 0; i < 256; i++)
            children[i] = nullptr;
    }
};
```

```
int numEntries(Node* p) {
    if (p == nullptr) return 0;
    int count = p->marked ? 1 : 0;
    for (int i = 0; i < 256; i++)
        count += numEntries(p->children[i]);
    return count;
}
```

11. Skyrim Loot Problem:

You are playing Skyrim and your character can carry a total weight of W in their inventory, where W is some positive integer. You enter the bandits' lair where you find n pieces of loot. The first piece of loot weighs w_1 pounds and has a value of v_1 gold pieces, where w_1 and v_1 are positive integers. The second item in the lair has weight w_2 and value v_2 , etc. Your goal is to select a subset of the n items to take such that the total value is as large as possible, but the sum of the weight is less or equal to your weight capacity W . In other words:

Input:

- a positive integer W
- n pairs of positive integers denoting the weight and value of each object: $(w_1, v_1), (w_2, v_2), \dots, (w_n, v_n)$

Output: The maximum possible total value you can obtain by selecting a subset of the n items while keeping the total weight of the selected items to be at most W .

Design an algorithm to solve the Skyrim loot problem. Analyze the run time of your solution in terms of W and n .

12. Zippers: Given three strings, you are to determine whether the third string can be formed by combining the characters in the first two strings. The first two strings can be mixed arbitrarily, but the characters from each must stay in their original order in the third string.

For example, consider forming "tcarete" from "cat" and "tree":

String A: **c a t**

String B: **t r e e**

String C: **t c a r e t e**

As you can see, we can form string C by selecting the first character of "tree", followed by the first 2 characters of "cat", followed by the second and third characters of "tree", followed by the last character of "cat" and "tree" respectively.

As a second example, consider forming "catrtee" from "cat" and "tree":

String A: **c a t**

String B: **t r e e**

String C: **c a t r t e e**

The answer for this input is also 'yes'

Finally, notice that it is impossible to form "cttaree" from "cat" and "tree", meaning the answer for this input would be 'no'.

Zipper Problem

Input

- String A = $a_1, a_2, a_3, \dots, a_n$
- String B = $b_1, b_2, b_3, \dots, b_m$
- String C = $c_1, c_2, c_3, \dots, c_{m+n}$

Output: Output *yes* if A and B can be combined (zippered) into string C.

Output *no* if A and B cannot be combined to form C.

Design an $O(nm)$ time algorithm to solve this problem.

13. Making Optimal Change

Consider the problem of making change for n cents using as few coins as possible.

- Describe a greedy algorithm to solve this problem using only quarters, dimes, nickels, and pennies. Prove that your algorithm will give the optimal (smallest) number of coins.
- Give a set of coins for which the greedy algorithm will not yield the optimal solution. Your set should include a penny so that a solution always exists.
- Give an $O(nk)$ -time algorithm to compute the optimal way to make change for n cents from any set of k -different coin denominations, assuming one of the denominations is a penny.

14. Santa needs some help

For this problem imagine you are Santa and you want to try to perfectly pack your sleigh with presents. That is, you want to pack up the sleigh with some of the presents from the workshop so that the total weight of the presents on the sleigh is exactly equal to the capacity of the sleigh, no more, no less. Formally, the problem is as follows:

Perfect Christmas Packing Problem:

Input

- n Christmas presents, each with weight $w_1, w_2, \dots, w_n$
- A positive integer k denoting the maximum weight Santa's sleigh can hold.

Output: Output *yes* if there is a subset of the n presents whose total weight sums to exactly k . Output *no* if there is no such subset.

Design a $O(nk)$ run-time algorithm to solve this problem.

15. Rocky Mountain High!

Suppose you are given an $n \times m$ matrix of positive integers denoting an elevation map of a park in the Rocky Mountains. Your goal is to design a path from the left side of the map (column 1) and the right side of the map (column m). Each step in the path must proceed by going either straight right to the next column, or diagonally up and right, or diagonally down and right (no going straight up or backwards). The cost of each step in the path is the absolute value of the difference in elevations between the two cells being traversed. The total cost of a path is the sum of each of step in the path. The goal is to compute a minimum possible cost path (hikers want a path that is as level as possible) connecting the left side of the map to the right side. Design an algorithm to compute the cost of the shortest possible path in $O(nm)$ time.

16. 4-Color is NP-complete

The 3-Color problem is a classic NP-complete problem in which you are given a graph and must output whether or not the graph is "3 colorable". A graph is k colorable if it is possible to assign each vertex one of k distinct colors such that no two adjacent vertices are the same color. The 4-color problem asks whether a given graph is 4-colorable.

- Show that the 4-color problem is also NP-complete.
- Show that the 100-color problem is also NP-complete.